

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-addf215b8409f4932.jsonl`  
- SHA-256: `afbdbc5a8434a2e439fc52ab09289ed1e371d379db9a5195deb82811238251e7`  
- Source modified: `2026-07-06T19:19:50+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:18:37.647Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (cross_adr): "Extra fields are ignored" means topic/security/deep_inspect config is SILENTLY DROPPED, not preserved ? save round-trip destroys it
severity high | section Schema ? minimal changes; Consequences ("Backwards-compatible")
summary: The ADR asserts config_json can "gain optional topics/allowlist/security_checks/deep_inspect keys ? all backwards-compatible with the existing FilterConfig Pydantic model (extra fields are ignored)." But Pydantic's default is extra='ignore', which DROPS unknown keys on parse. Every load_filter_config -> save_filter_config round trip (any /set command) will silently erase the new pipeline config.
why_it_matters: This is the load-bearing claim for the ADR's headline "no schema migration / backwards compatible" design. It is false as stated and would cause data loss for F2/F3/F4/F5 config the moment any existing /set command runs. The whole persistence story for the pipeline is broken unless FilterConfig is changed (model_config extra='allow' + typed sub-models, or the pipeline stores config outside FilterConfig).
evidence: Verified with .venv python: FilterConfig(min_duration_s=10, topics={...}, allowlist=[...], deep_inspect=True) parses OK but model_dump()/model_dump_json() emit ONLY the 9 known keys ? topics/allowlist/deep_inspect are gone; hasattr(fc,'topics')==False; FilterConfig.model_config == {} (i.e. default extra='ignore'). The save path proves the loss: db.py:1708 load = FilterConfig.model_validate_json(policy.config_json); db.py:1720 save = config_json=cfg.model_dump_json(). Load-then-save strips any key FilterConfig doesn't declare. models.py FilterConfig has no extra='allow' and no topics/allowlist/security_checks/deep_inspect fields.
rec: Change the claim to reflect reality and specify the mechanism: either (a) set model_config = ConfigDict(extra='allow') on FilterConfig AND stop round-tripping through it in the shim (or persist extras), or (b) add explicit typed fields (topics, allowlist, security_checks, deep_inspect) to FilterConfig ? which is a model change, not "extra fields are ignored, no migration." Note that F2-F5 have a hard dependency on this fix that the phasing table omits.

2. assistant (2026-07-06T19:18:40.857Z)

I'll verify this finding by reading the ADR section, checking the code, and running Python to test the Pydantic behavior.

3. user (2026-07-06T19:18:43.038Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- ****Status****: Proposed
- ****Date****: 2026-07-06
- ****Decision owner****: project owner
- ****Tracking issue****: #31 (Track F of the provider-neutral umbrella, #25)
- ****Depends on****: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is ****flat****: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a ****policy pipeline**** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

```
| Concept | Current state |
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |
```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                       ?
?                               ?                                     ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                   ?
?   duration, resolution, size, codec, mime, GIF/round              ?
?   ? extends today's FilterEngine                                  ?
?                               ?                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)                ?
?   source allowlist, keyword match, channel topic tags            ?
?   ? user-expressed interests filter the stream                   ?
?                               ?                                     ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)           ?
?   URL/domain reputation, file metadata sanity,                   ?
?   caption text scanning                                           ?
?   ? cheap checks; no download                                     ?
?                               ?                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                      ?
?   ffprobe, perceptual hash, downloaded content scan              ?
?   ? gated behind explicit user opt-in + budget                   ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
```

```

 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...

```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```

```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
```
```

The caller (telethon\_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

## Schema ? minimal changes

**\*\*No schema migration\*\*** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

## Implementation phasing (future issues)

| Phase   | Scope                   | Dependencies                                                                  |
|---------|-------------------------|-------------------------------------------------------------------------------|
| ***F1** | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler |
| ***F2** | Topic preferences       | `topics` config parsing, source title/caption keyword matching                |
| ***F3** | Source allowlist        | `allowlist` config, skip non-allowlisted sources                              |
| ***F4** | URL/domain reputation   | Domain blacklist, caption link extraction                                     |
| ***F5** | Deep inspection         | ffprobe/perceptual hash, download budget enforcement                          |
| ***F6** | Policy decisions table  | Audit trail storage, explainability UI                                        |
| ***F7** | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace                |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **\*\*design contract\*\*** the phases code against.

## Consequences

### **\*\*Positive\*\***

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

## **\*\*Negative\*\***

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

## **Alternatives considered**

- **\*\*Do nothing; keep flat FilterConfig.\*\*** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **\*\*Inline each stage in the handler\*\*** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **\*\*Use a rules engine / DSL.\*\*** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **\*\*Bundle all phases into a single PR.\*\*** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

## **Open questions for the decision owner**

1. **\*\*`StageResult.metadata` shape\*\*** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **\*\*Policy decisions table\*\*** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **\*\*Topic matching\*\*** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **\*\*Deep inspection budget\*\*** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

### **4. assistant (2026-07-06T19:18:45.635Z)**

Now let me verify the two key claims: (1) the ADR's assertion about extra fields, and (2) the actual save round-trip code path.

### **5. user (2026-07-06T19:18:49.786Z)**

parsed OK

has topics attr: False

model\_config: {}

dump: {"min\_duration\_s":5,"max\_duration\_s":7200,"target\_height":1080,"max\_size\_mb":2048,"exclude\_gif":true,"exclude\_round":true,"exclude\_nosound":false,"delivery\_mode":"forward","delivery\_target":"me"}

### **6. assistant (2026-07-06T19:18:52.561Z)**

The Python check confirms: extra keys parse without error but are dropped from `model\_dump\_json()`. Now let me verify the save round-trip code path in db.py.

### **7. user (2026-07-06T19:18:54.467Z)**

```

83: config_json TEXT NOT NULL
132: config_json TEXT NOT NULL,
476: config_json TEXT NOT NULL
479: "user_id, config_json",
622: INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
624: SELECT user_id, 'default', config_json, 1,
695: CASE WHEN json_valid(config_json)
696: THEN json_extract(config_json, '$.delivery_target') END,
701: CASE WHEN json_valid(config_json)
702: THEN json_extract(config_json, '$.delivery_target') END,
726: CASE WHEN json_valid(fcd.config_json)
727: THEN json_extract(fcd.config_json, '$.delivery_target') END,
885: config_json=row["config_json"],
1253: """INSERT INTO policies (account_id, name, config_json, is_default,
1257: config_json=excluded.config_json,
1263: policy.config_json,
1698:async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1708: return FilterConfig.model_validate_json(policy.config_json)
1711:async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
1720: config_json=cfg.model_dump_json(),

```

## 8. user (2026-07-06T19:19:02.277Z)

```

async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
 """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.

 ADR 0011 (Option 2A): ``filter_config`` is renamed to
 ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
 instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
 """
 policy = await get_default_policy(conn, user_id)
 if policy is None:
 return FilterConfig()
 return FilterConfig.model_validate_json(policy.config_json)

async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg: FilterConfig) ->
None:
 """Deprecated shim: upsert the account's default :class:`Policy`."""
 existing = await get_default_policy(conn, user_id)
 await upsert_policy(
 conn,
 Policy(
 id=existing.id if existing is not None else None,
 account_id=user_id,
 name="default",
 config_json=cfg.model_dump_json(),
 is_default=True,
 created_at=(existing.created_at if existing is not None else _now()),
 updated_at=_now(),
),
)

```

## --- bulk helpers (used by the manager on startup) -----

```

async def list_videos_for_user(# v2: used by on-demand /search; not called in v1
 conn: aiomysqlite.Connection, user_id: int, limit: int = 100
) -> list[VideoRecord]:
 """Deprecated shim: list the account's content items projected onto VideoRecord."""
 conn.row_factory = aiomysqlite.Row

```

## 9. assistant (2026-07-06T19:19:06.183Z)

Confirmed. `load\_filter\_config` at db.py:1708 parses via `FilterConfig.model\_validate\_json`, and `save\_filter\_config` at db.py:1720 serializes via `cfg.model\_dump\_json()`. Any load?modify?save cycle strips extras.

Now let me adversarially check whether a `/set` command actually goes through this round-trip (does it load, mutate, and save via FilterConfig?), which is the mechanism of data loss.

## 10. user (2026-07-06T19:19:07.143Z)

```
src\tg_video_filter\commands.py:78: current_cfg = await db.load_filter_config(conn,
user.id)
src\tg_video_filter\commands.py:82: new_cfg = await db.load_filter_config(conn,
user.id)
src\tg_video_filter\commands.py:112: cfg = await db.load_filter_config(conn, user_id)
src\tg_video_filter\bootstrap.py:75: await db.save_filter_config(conn, user_id,
DEFAULT_FILTER_CONFIG)
src\tg_video_filter\db.py:618: # (Option 2A). is_default=1 so the legacy
load_filter_config shim reads
src\tg_video_filter\db.py:1698:async def load_filter_config(conn: aiomysqlite.Connection, user_id:
int) -> FilterConfig:
src\tg_video_filter\db.py:1711:async def save_filter_config(conn: aiomysqlite.Connection, user_id:
int, cfg: FilterConfig) -> None:
src\tg_video_filter\db.py:1809: "load_filter_config",
src\tg_video_filter\db.py:1810: "save_filter_config",
src\tg_video_filter\manager.py:140: cfg = await db.load_filter_config(conn,
session.account_id)
src\tg_video_filter\models.py:193: ``load_filter_config`` / ``save_filter_config`` shims
read/write that row.
src\tg_video_filter\command_service.py:465: await db.save_filter_config(conn, user_id,
cfg)
```

## 11. user (2026-07-06T19:19:11.627Z)

```
max_size_mb: int | None = None,
max_duration_s: int | None = None,
min_duration_s: int | None = None,
target_height: int | None = None,
exclude_gif: bool | None = None,
exclude_round: bool | None = None,
exclude_nosound: bool | None = None,
) -> FilterConfig:
 """Apply partial updates to the account's config, persist, and return."""
 updates: dict[str, object] = {}
 if max_size_mb is not None:
 updates["max_size_mb"] = max_size_mb
 if max_duration_s is not None:
 updates["max_duration_s"] = max_duration_s
 if min_duration_s is not None:
 updates["min_duration_s"] = min_duration_s
 if target_height is not None:
 updates["target_height"] = target_height
 if exclude_gif is not None:
 updates["exclude_gif"] = exclude_gif
 if exclude_round is not None:
 updates["exclude_round"] = exclude_round
 if exclude_nosound is not None:
 updates["exclude_nosound"] = exclude_nosound
 cfg = current.model_copy(update=updates) if updates else current
 await db.save_filter_config(conn, user_id, cfg)
 log.info(
 "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
 "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
 user_id,
```



```
cfg.max_size_mb,
```

## 12. assistant (2026-07-06T19:19:14.034Z)

Let me see where `current` comes from in this `/set` path to confirm it's a load?copy?save cycle.

## 13. user (2026-07-06T19:19:15.572Z)

```
Try by title (case-insensitive substring match).
all_srcs = await db.list_account_sources(conn, user_id)
target_lower = target.lower()
for s in all_srcs:
 if s.title and target_lower in s.title.lower():
 return s
return None

-- internal -----

async def _update(
 self,
 conn: aiosqlite.Connection,
 user_id: int,
 current: FilterConfig,
 max_size_mb: int | None = None,
 max_duration_s: int | None = None,
 min_duration_s: int | None = None,
 target_height: int | None = None,
 exclude_gif: bool | None = None,
 exclude_round: bool | None = None,
 exclude_nosound: bool | None = None,
) -> FilterConfig:
 """Apply partial updates to the account's config, persist, and return."""
 updates: dict[str, object] = {}
```

## 14. user (2026-07-06T19:19:24.271Z)

```
230: cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
250: cfg = await self._update(conn, user_id, current_cfg, max_duration_s=new_secs)
270: cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
282: cfg = await self._update(conn, user_id, current_cfg, target_height=new_height)
305: cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
435: async def _update(
467: "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
```

## 15. user (2026-07-06T19:19:27.375Z)

```
text = text.rstrip()

--- /set size <value><unit> -----
m = _SIZE_RE.match(text)
if m:
 value_str, unit = m.group(1), m.group(2) or ""
 # NOTE: ValueError is unreachable here ? the regex already validates
 # the unit against [kmgT]?b?, so parse_size_mb can never raise.
 # Kept for defensive parity with the original commands.py refactor.
 try:
 new_mb = parse_size_mb(value_str, unit)
 except ValueError:
 return CommandResult("? Invalid size format. Use e.g. `/set size 10MB`.")
 if new_mb < 1:
 return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
```

```
cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
return CommandResult(
 f"? Max file size set to **{_format_size(new_mb)}**.\n\n{format_config(cfg)}",
```

#### 16. assistant (2026-07-06T19:19:37.221Z)

Confirmed the full chain. `current\_cfg` is loaded via `load\_filter\_config` (which parses through `FilterConfig`, dropping extras), then `model\_copy` + `save\_filter\_config` (which serializes only the 9 known keys). So even if extras somehow reached the DB, the very first `/set` command destroys them.

Let me do one adversarial check: does the ADR anywhere qualify this, e.g. saying pipeline config is stored outside FilterConfig, or that the shim would be replaced? Re-reading the Schema section: it explicitly says config gains those keys "all backwards-compatible with the existing FilterConfig Pydantic model (extra fields are ignored)." The phrase "extra fields are ignored" is presented as the mechanism that makes it safe ? but "ignored" in Pydantic means \*dropped on dump\*, which is precisely the data-loss vector. The ADR treats "ignored" as "harmlessly preserved," which is false.

One more refutation attempt: is it possible the pipeline reads `config\_json` directly (raw JSON) rather than through FilterConfig, so it never round-trips? The ADR's F2/F3 phases say "topics config parsing" and "allowlist config" ? but persistence still flows through `save\_filter\_config`/`FilterConfig` for any `/set`. The ADR does not propose a separate persistence path, nor exempt these keys from the shim. So the gap is real and unaddressed.

The finding holds. Severity: the load-bearing "no migration / backwards-compatible" claim is false, and it causes silent data loss for F2-F5 config. That's a high-severity design inaccuracy for a design contract ? the entire persistence story is broken as written.

#### 17. user (2026-07-06T19:19:50.181Z)

Structured output provided successfully